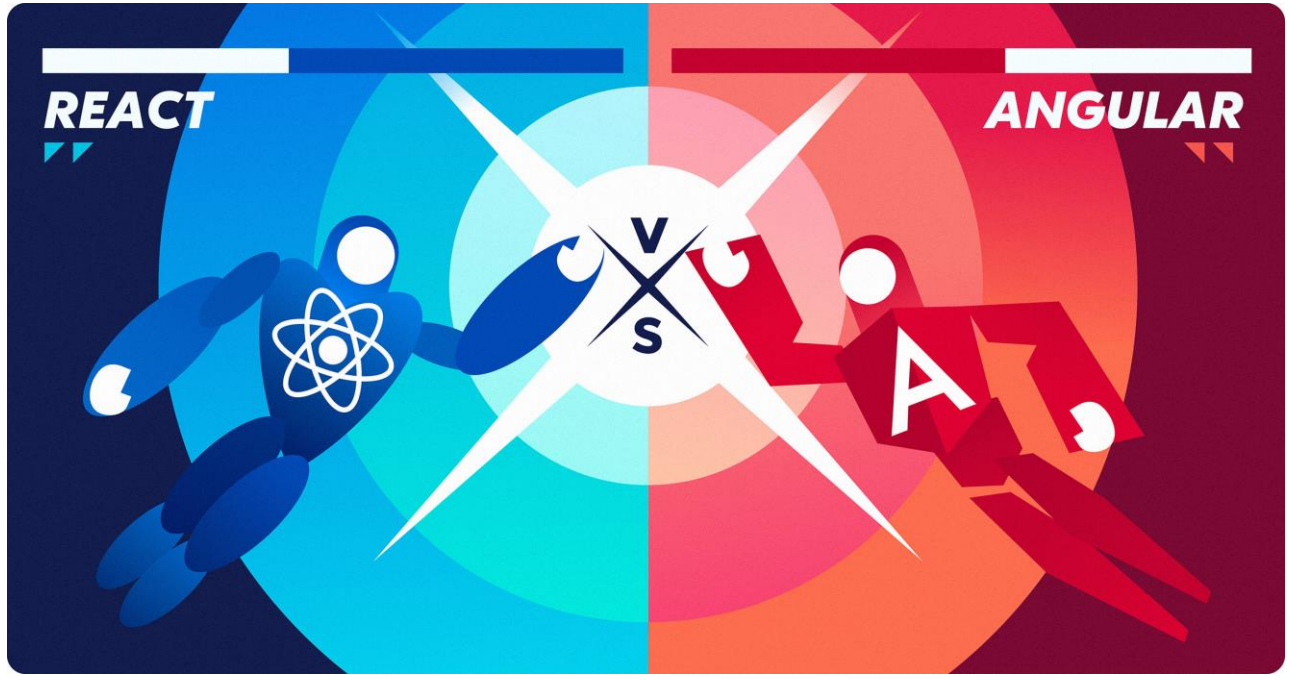

TECHNOLOGIES WEB II (React & Angular)

Professeur	Dr. Fadwa LACHHAB
Niveau	4eme année
Année	2024-2025

Partie 2 : Angular

Semaine 1	Semaine 2	Semaine 3	Semaine 4	Semaine 5	Semaine 6
Presentation de Angular Environnement de développement Components	Data Binding/Directives	Forms/ Services Routing	HTTP/Observables Json Server	Interaction avec Base de Données : FireBase	Examen TP

LE PROTOCOLE HTTP

Forms

- Dans **Angular** à partir de la version 2, voir jusqu'à la version 7 et même après, on peut distinguer deux types de création des formulaires.
- **Template-Driven Forms :** souvent est la méthode la plus utilisée pour la création et la gestion des formulaires.
- Cette approche est appelée l'approche de **AngularJS**. Car c'est la même utilisée dans AngularJS. Cela ne signifie pas qu'elle est moins bonne.
- **Reactive Forms:** C'est la moins utilisées mais la plus façonnable et flexible.

LE PROTOCOLE HTTP

- **HTTP :HyperText Tranfert Protocol**
  - Protocole qui permet au client de récupérer des documents du serveur
  - Ces documents peuvent être statiques (contenu qui ne change pas : HTML, PDF, Image, etc..) ou dynamiques (Contenu généré dynamiquement au moment de la requête : PHP, JSP, ASP…)
  - Ce protocole permet également de soumissionner les formulaires
- **Fonctionnement (très simple en HTTP/1.0)**
  - Le client se connecte au serveur (Créer une socket)
  - Le client demande au serveur un document : Requête HTTP
  - Le serveur renvoi au client le document (status=200) ou d'une erreur (status=404 quand le document n'existe pas)
  - Déconnexion

Méthodes du protocole HTTP

- Une requête HTTP peut être envoyée enutilisant les méthodes suivantes:
  - GET : Pour récupérer le contenu d'un document
  - POST : Pour soumissionner des formulaires (Envoyer,dans la requête, des données saisies par l'utilisateur)
  - PUT pour envoyer un fichier du client vers le serveur
  - DELETE permet de demander au serveur de supprimer un document.
  - HEAD permet de récupérer les informations sur un document (Type, Capacité, Date de dernièremodification etc…)

HTTP: interaction avec le back end

- La plupart des applications communiquent avec les services backend via le protocole **HTTP**;

- **HttpClient** du module **@angular/common/http**, offre une API HTTP client simplifiée pour les applications Angular;

- Les avantages supplémentaires de **HttpClient** inclus des fonctionnalités de test, des objets de requêtes et de réponse typés, un mécanisme d'interception de requêtes et de réponse, des apis observables et une gestion simplifiée des erreurs.

Exemple de service avec HttpClient

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable()
export class ExampleService {

  private host = 'http://localhost:8080';

  constructor(private http: HttpClient) { }

  getTasks() {
    return this.http.get(this.host + '/tasks');
  }

  login(_user) {
    return this.http.post(this.host + '/login', _user, { observe: 'body' });
  }

  saveTask(_task) {
    return this.http.post(this.host + '/tasks',
      _task,
      {
        headers: new HttpHeaders({ autorization: 'my-token' })
      });
  }

}
```


UNIVERSIAPOLIS Université Internationale d'Agadir الجامعة الدولية لأكادير
ECOLE POLYTECHNIQUE D'AGADIR

HttpClientModule

- Pour utiliser **HttpClient**, vous devez importer le module **HttClientModule** de Angular;
- La plupart du temps, cela se fait dans le module racine **AppModule**.

```
import { AboutComponent } from './about/about.component';
import { ContactsComponent } from './contacts/contacts.component';
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';
import { ExampleService } from './services/example/example.service';
import { AboutService } from './services/about/about.service';

@NgModule({
  declarations: [
    AppComponent,
    AboutComponent,
    ContactsComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule,
    HttpClientModule
  ],
  providers: [ExampleService, AboutService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Exemple dans un composant

```
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { ExampleService } from '../services/example/example.service';

@Component({
  selector: 'app-task',
  templateUrl: './task.component.html',
  styleUrls: ['./task.component.css']
})
export class TaskComponent implements OnInit {

  private tasks;

  constructor(public exampleService: ExampleService, private router: Router) { }

  ngOnInit(): void {
    this.exampleService.getTasks()
      .subscribe(data => {
        this.tasks = data;
      }, err => {
        console.log('Error...');
        this.router.navigateByUrl('/about');
      });
  }

}
```

Json Server

Installer JSON Server

- Installez JSON Server globalement avec npm :

```
npm install -g json-server
```

- Ou ajoutez-le comme dépendance dans votre projet Angular :

```
npm install json-server --save-dev
```

Créer un fichier de données JSON

- À la racine de votre projet Angular, créez un fichier appelé db.json

- Définissez vos données. Par exemple, pour une liste d'employés

```
{
  "employees": [
    { "id": 1, "name": "Alice", "position": "Developer" },
    { "id": 2, "name": "Bob", "position": "Designer" },
    { "id": 3, "name": "Charlie", "position": "Manager" }
  ]
}
```

Lancer JSON Server

- Pour démarrer JSON Server, utilisez la commande suivante :

json-server --watch db.json

- JSON Server sera accessible à l'adresse :

http://localhost:3000

Lancer JSON Server

Exemple de routes générées :

- GET **http://localhost:3000/employees** : Récupère la liste des employés.
- POST **http://localhost:3000/employees** : Ajoute un employé.
- PUT **http://localhost:3000/employees/:id** : Met à jour un employé.
- DELETE **http://localhost:3000/employees/:id** : Supprime un employé.

Configurer Angular pour utiliser JSON Server

- Installez le module HTTP d'Angular si ce n’est pas déjà fait :

ng add @angular/common/http

- Ajoutez un service Angular pour interagir avec l'API
- Implémentez les appels HTTP dans le service
- Utiliser le service dans un composant Angular

Observables

Observable

Un Observable est un objet qui émet des données au fil du temps. Il est utilisé pour gérer des opérations asynchrones telles que les requêtes **HTTP**, les événements utilisateur ou les WebSockets. Avec Angular, les Observables sont fournis par la bibliothèque **RxJS**

Les Observables avec HTTP dans Angular

Angular utilise le module **HttpClient** pour effectuer des requêtes **HTTP**.

Les méthodes de **HttpClient** (comme get, post, etc.) renvoient des Observables, ce qui permet de **souscrire** à ces flux de données et de réagir lorsqu'une réponse est reçue.

Étapes générales pour utiliser les Observables dans un service

- Importer **HttpClientModule** dans le module principal.
- Créer un **service** Angular pour gérer les **requêtes HTTP**.
- Utiliser les méthodes de **HttpClient** pour effectuer des requêtes.
- **Souscrire** à l'Observable dans le composant pour recevoir les données.

Étapes générales pour utiliser les Observables dans un service

service.ts

```
export class EmployeeService {
  private apiUrl = 'http://localhost:3000/employees';
  constructor(private http: HttpClient) {}
  getEmployees(): Observable<any[]> {
    return this.http.get<any[]>(this.apiUrl);
  }
```

- **apiUrl** : Définit l'URL de l'API REST à interroger.
- **HttpClient** : Fournit les outils pour effectuer des requêtes HTTP.
- **getEmployees** : Méthode qui renvoie un observable pour récupérer les données des employés via une requête **HTTP GET**.

Étapes générales pour utiliser les Observables dans un service

component.ts

```
public getEmployees() {
  this.employeeService.getEmployees().subscribe((data) => {
    this.employees = data;
  });
}
```

- La méthode **getEmployees**() est appelée (par exemple, dans **ngOnInit**()).
- Elle utilise le service **employeeService** pour effectuer une requête HTTP et récupérer les employés.
- Une fois les données reçues (grâce à **subscribe**), elles sont stockées dans la variable **employees**.
- Ces données peuvent ensuite être utilisées dans le composant, par exemple pour être affichées dans une table HTML.

Cas pratique

Création d'un nouveau projet

- On va créer un nouveau projet

ng new employes

- On va installer le bootstrap dans notre application

npm install bootstrap --save

- pour ajouter le bootstrap à notre projet on doit ajouter cette ligne dans le fichier angular.json

```
"styles": [
"node_modules/bootstrap/dist/css/bootstrap.css",
"src/styles.scss"
],
```

Configuration des routes

- On va créer un component pour afficher la liste des employers
- ng g c employees/list/listEmployes
- On va créer un component pour la création d'un nouveau employer.
- ng g c employees/create/createEmploye
- Puis on va ajouter les routes nécessaires pour naviguer entre les components.

➔ Pour cela on doit modifier le fichier app.module.ts

Configuration des routes

TS employee.service.ts TS *app-routing.module.ts* ×

src > app > TS app-routing.module.ts > ...

```
1  import { NgModule } from '@angular/core';
2  import { RouterModule, Routes } from '@angular/router';
3  import { CreateEmployeeComponent } from './employees/create/create-employee/
4  import { ListEmployeesComponent } from './employees/list/list-employees/list
5
6  const routes: Routes = [
7    { path: 'list', component: ListEmployeesComponent },
8    { path: 'create', component: CreateEmployeeComponent },
9    { path: '', redirectTo: '/list', pathMatch: 'full' }
10
11 ];
12
```

Configuration des routes

- Dans le fichier **app.component.html** on doit ajouter le nav bar pour assurer la navigation entre les deux composants.
- Aussi il faut ajouter le **<router-outlet>**

TS employee.service.ts × <> app.component.html ●

src > app > <> app.component.html > div.container

```
1   <div class="container">
2     <nav class="navbar navbar-default">
3       <ul class="nav navbar-nav">
4         <li>
5           <a routerLink="list">Liste Employes</a>
6         </li>
7         <li>
8           <a routerLink="create">Ajouter Employe</a>
9         </li>
10      </ul
11    </nav>
12    <router-outlet></router-outlet>
13  </div>
```

Création de json serveur et du serveur Data

TS employee.service.ts | app.component.html | {} Employes.json ×

{} Employes.json > [ ] employees > {} 1

```
1   {
2       "employees": [
3         {
4           "id": 1,
5           "nom": "Ahmed",
6           "prenom": "Latif"
7         },
8         {
9           "id": 2,
10          "nom": "Amine",
11          "prenom": "Sousse"
12        },
13        {
14          "id": 3,
15          "nom": "Manel",
16          "prenom": "Benchahb"
17        }
18      ]
19  }
```

Création de json serveur et du serveur Data

- On va démarrer le serveur par la commande suivante:

json-server --watch Employes.json

- Le json serveur est démarrer sur le port 3000 et l'URL suivante: http://localhost:3000/employees

localhost:3000/employees

screen shot 2018-0... Google Boîte de réceptio

Impression élégante


```
[
  {
    "id": "1",
    "nom": "Ahmed",
    "prenom": "Latif"
  },
  {
    "id": "2",
    "nom": "Amine",
    "prenom": "Sousse"
  },
  {
    "id": "3",
    "nom": "Manel",
    "prenom": "Benchahb"
  }
]
```

Installation du HttpClient

- On doit ajouter le http module dans le **app.module.ts**

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { ListEmployeesComponent } from './employees/list/list-employees/lis
import { CreateEmployeeComponent } from './employees/create/create-employee
import { HttpClientModule } from '@angular/common/http';
@NgModule({
  declarations: [
    AppComponent,
    ListEmployeesComponent,
    CreateEmployeeComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule
  ],
```

Création du service

- Dans notre projet employer on va créer **employee.service.ts**

TS employee.service.ts × | <> app.component.html | TS app.module.ts

src > app > services > TS employee.service.ts > EmployeeService > deleteEmployee

```
1  import { HttpClient } from '@angular/common/http';
2  import { Injectable } from '@angular/core';
3  import { Observable } from 'rxjs';
4
5  @Injectable({
6    providedIn: 'root'
7  })
8  export class EmployeeService {
9    private apiUrl = 'http://localhost:3000/employees';
10   constructor(private http: HttpClient) {}
11   getEmployees(): Observable<any[]> {
12     return this.http.get<any[]>(this.apiUrl);
13   }
14
```

Injection dans le composant

- Puis la création du fichier **list-employees.component.ts**

TS employee.service.ts | TS list-employees.component.ts × | <> app.component.html

src > app > employees > list > list-employees > TS list-employees.component.ts > ListEmployeesC

```
1   import { Component } from '@angular/core';
2   import { EmployeeService } from 'src/app/services/employee.service';
3
4   @Component({
5     selector: 'app-list-employees',
6     templateUrl: './list-employees.component.html',
7     styleUrls: ['./list-employees.component.css']
8   })
9   export class ListEmployeesComponent {
10    employees: any[] = [];
11    constructor(private employeeService: EmployeeService) { }
12    ngOnInit() {
13      this.getEmployees();
14    }
15    public getEmployees() {
16      this.employeeService.getEmployees().subscribe((data) => {
17        this.employees = data;
18      });
19    }
20  }
```

Affichage

- Puis pour la partie affichage on va modifier le fichier **list-employees.component.html**

```
<table class="table table-bordered">
    <thead>
      <tr>
        <th>Id</th>
        <th>Prenom</th>
        <th>Nom</th>
      </tr>
    </thead>
    <tbody>
      <tr *ngFor="let employee of employees">
        <td>{{employee.id}}</td>
        <td>{{employee.nom}}</td>
        <td>{{employee.prenom}}</td>
      </tr>
    </tbody>
  </table>
```

Lancement du projet

- En fin on fait le build et on lance le serveur par la commande ng serve et dans le navigateur avec l'adresse suivante: http://localhost:4200/ on affiche la page suivante

Gestion Employés Liste Employés Ajouter Employé

Id	Prenom	Nom
1	Ahmed	Latif
2	Amine	Sousse
3	Manel	Benchahb